

02 - Authentication and Progressive Onboarding

Feature objective

Get the user from install to a meaningful first financial plan with the smallest amount of required input, while preserving progress and never forcing SMS permission.

Dependencies

- Foundation/App Shell
- backend auth endpoints
- shared SDK

In scope

Authentication:

- register
- login
- Google login
- forgot/reset password
- email verification where backend requires it
- auth boot/restore
- logout

Onboarding:

- welcome
- initial goal selection
- income
- fixed obligations
- approximate balances/savings
- loans/EMIs
- SMS explanation/permission
- 60-day initial SMS scan
- initial financial review
- initial plan generation
- first-plan summary
- resume interrupted onboarding

Out of scope

- complete household management
- full KYC
- live bank linking
- iOS onboarding parity
- PDF statement import

Authentication model

Mobile auth flow:

Login/Register

v

backend returns access + refresh

v

access token -> memory

refresh token -> SecureStore

v

API transport

```

    v
401 -> refresh -> retry once
On boot:
Read refresh token
    v
refresh session
    v
fetch /me
    v
route based on onboarding completion
If refresh fails, clear local auth state and show Login.

```

Routes

```

(auth)/
|-- login.tsx
|-- register.tsx
|-- forgot-password.tsx
|-- reset-password.tsx
|-- verify-email.tsx
`-- oauth-callback.tsx

```

```

(onboarding)/
|-- index.tsx
|-- goals.tsx
|-- income.tsx
|-- obligations.tsx
|-- balances.tsx
|-- loans.tsx
|-- sms-intro.tsx
|-- sms-import.tsx
|-- review.tsx
|-- generating.tsx
`-- ready.tsx

```

Feature folder

```

features/auth/
|-- screens/
|-- components/
|-- context/
|   `-- AuthContext.tsx
|-- hooks/
|-- services/
|-- schemas/
`-- types.ts

```

```

features/onboarding/
|-- screens/
|-- components/
|-- forms/
|-- hooks/
|-- services/
|-- store/
|-- schemas/
`-- types.ts

```

Onboarding principle

Ask only enough to generate a useful first plan.

Required baseline information:

- take-home income
- fixed obligations
- approximate savings/current balances
- loans/EMIs
- up to three goals

Everything else can be progressively collected later.

Goal-first onboarding

First meaningful question:

What do you want your money to achieve?

Presets:

- emergency fund
- home
- vehicle
- travel
- education
- marriage
- clear debt
- retirement
- build wealth
- custom

Allow maximum 3 active goals.

At this stage, selecting a goal does not require full configuration.

Income step

Collect:

- monthly take-home income
- frequency
- stability
- optional additional recurring income

Keep the screen amount-first and simple.

Obligations step

Collect recurring must-pay items:

- rent
- family contribution
- utilities
- subscriptions
- insurance
- other fixed expenses

Allow skip and later edit.

Balances step

Collect approximate:

- bank savings
- cash
- current investments

UI must explicitly say estimates are acceptable.

Loan step

For each loan if known:

- type
- outstanding amount
- EMI
- interest rate optional
- remaining tenure optional

Do not block onboarding because the user does not know an interest rate.

SMS permission step

Never immediately open the Android permission dialog.

First show a rationale screen:

- what is scanned
- why it improves the product
- non-financial messages are ignored
- raw personal conversations are not uploaded
- normalized financial records are synced
- manual mode remains available

Primary action: Enable SMS sync

Secondary: I will add transactions manually

For MVP foreground scanning, request only the permission actually required for reading SMS history. Do not request background receive permission unless the product later introduces a real background receiver.

First import

Default scope: last 60 days.

Display real progress and result counts:

```
financial messages found
transactions detected
items needing review
messages ignored
```

Never expose a raw SMS inbox UI.

Review step

Before generating the first plan, show a compact financial summary:

- inferred/entered income
- average spending
- total EMI
- approximate tracked savings
- number of transactions
- unresolved review count

Allow:

- Looks right

- Review transactions
- Continue with warnings where safe

Plan generation step

Use backend progress events/SSE if available.

Show real stages:

Building financial snapshot
Evaluating goals
Checking cash-flow capacity
Generating roadmap

If generation fails, preserve onboarding data and offer Retry. Do not force the user to re-enter data.

First plan result

Show only the high-signal summary:

- Financial Health Score
- monthly surplus
- number of goals on track
- most important next action

Actions:

- Go to my plan
- See dashboard

Onboarding persistence

Persist onboarding progress server-side when practical. Keep enough local progress to resume a partially completed form even if the last server write has not happened yet.

Store:

- current step
- completed steps
- local draft values
- SMS permission decision
- first import state

Do not store secrets or raw SMS bodies in generic persisted state.

API requirements

Conceptual endpoints:

POST /auth/register
POST /auth/login
POST /auth/refresh
POST /auth/logout
GET /auth/me

GET /onboarding/status
PATCH /onboarding/profile
PATCH /onboarding/goals
PATCH /onboarding/finances
POST /onboarding/complete
POST /plans/generate-initial

Exact paths should follow backend contracts rather than mobile inventing separate semantics.

TanStack Query hooks

Examples:

```
useMe
useOnboardingStatus
useSaveOnboardingStep
useGenerateInitialPlan
```

AuthContext owns current user/auth lifecycle; TanStack Query owns ordinary server data.

UX states

Must handle:

- offline login failure
- invalid credentials
- unverified email
- Google OAuth cancellation
- SMS denied
- SMS permanently denied
- no financial SMS found
- incomplete transaction review
- plan generation failure
- onboarding interrupted and resumed

Step-by-step implementation

1. Implement AuthContext and token lifecycle.
2. Build login/register/reset screens.
3. Implement boot restore and routing guards.
4. Create onboarding state contract and resume logic.
5. Build goal selection.
6. Build income/obligations/balances/loans forms with RHF + Zod.
7. Build SMS rationale screen and hand off to SMS feature.
8. Build financial review screen.
9. Integrate initial-plan generation.
10. Build first-plan summary.
11. Add failure/retry/resume states.

Acceptance criteria

- user can register/login/logout and restore session after app restart
- user never loses completed onboarding steps after app restart
- SMS denial does not block onboarding
- app can complete onboarding using manual financial data only
- user can import 60 days of SMS when permission is granted
- initial plan generation starts automatically after review
- generation failure is recoverable
- completion routes user into Home/Plan without another setup gate

Tests

- auth refresh + logout
- boot route decision
- onboarding resume
- form validation
- SMS denial fallback
- initial-plan retry state
- user can finish onboarding with zero detected SMS